

MindSpore Transformers 大模型系列课程

MindSpore Transformers LLM数据预处理介绍与实践

主讲人：牛君豪

昇思MindSpore AI工程师

MindSpore Transformers SIG Contributor

专题一：套件介绍

《MindSpore Transformers训推Mcore架构介绍》

《MindSpore Transformers环境安装与镜像制作》

专题二：LLM训练

《MindSpore Transformers LLM预训练与微调介绍与实践》

《MindSpore Transformers LLM数据预处理介绍与实践》

《MindSpore Transformers Safetensors权重详解》

《MindSpore Transformers断点续训介绍与实践》

《MindSpore Transformers训练在线监控介绍与实践》

《MindSpore Transformers训练高可用特性介绍》

专题三：LLM推理

《MindSpore Transformers推理介绍与实践》

《MindSpore Transformers & vLLM部署与评测》

专题四：高阶开发

《MindSpore Transformers LLM推理迁移与实践》

《MindSpore Transformers & vLLM推理精度比对》

《MindSpore Transformers LLM训练迁移与实践》

《MindSpore Transformers & Megatron-LM训练精度比对》

《MindSpore Transformers训练性能调优》

《MindSpore Transformers推理性能调优》

《MindSpore Transformers DeepSeek-R1蒸馏实践》

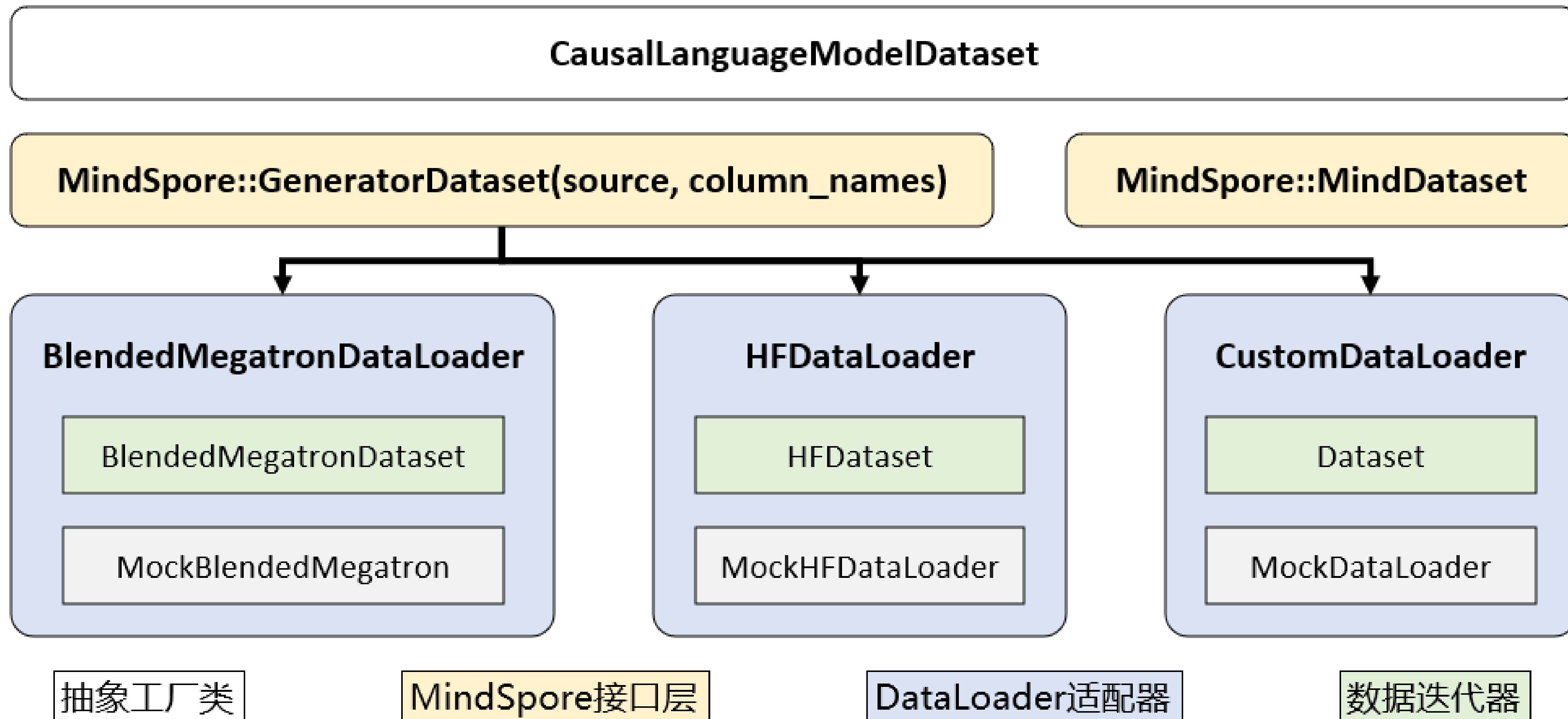
目录

- 1. MindSpore Transformers数据集功能概述**
- 2. 使用Megatron-LM数据集进行预训练的介绍与实践**
- 3. 使用Hugging Face数据集进行微调的介绍与实践**
- 4. MindSpore Transformers构造自定义数据集介绍**

1. MindSpore Transformers数据集功能概述

MindSpore Transformers数据集功能概述

MindSpore Transformers数据集组件架构图



◆MindDataset:

支持原生MindRecord格式数据加载，适用于预训练、微调场景；

◆Blended Megatron DataLoader:

在Megatron-LM BlendedDataset功能基础上进行改进，面向大模型预训练的多源混合数据集；

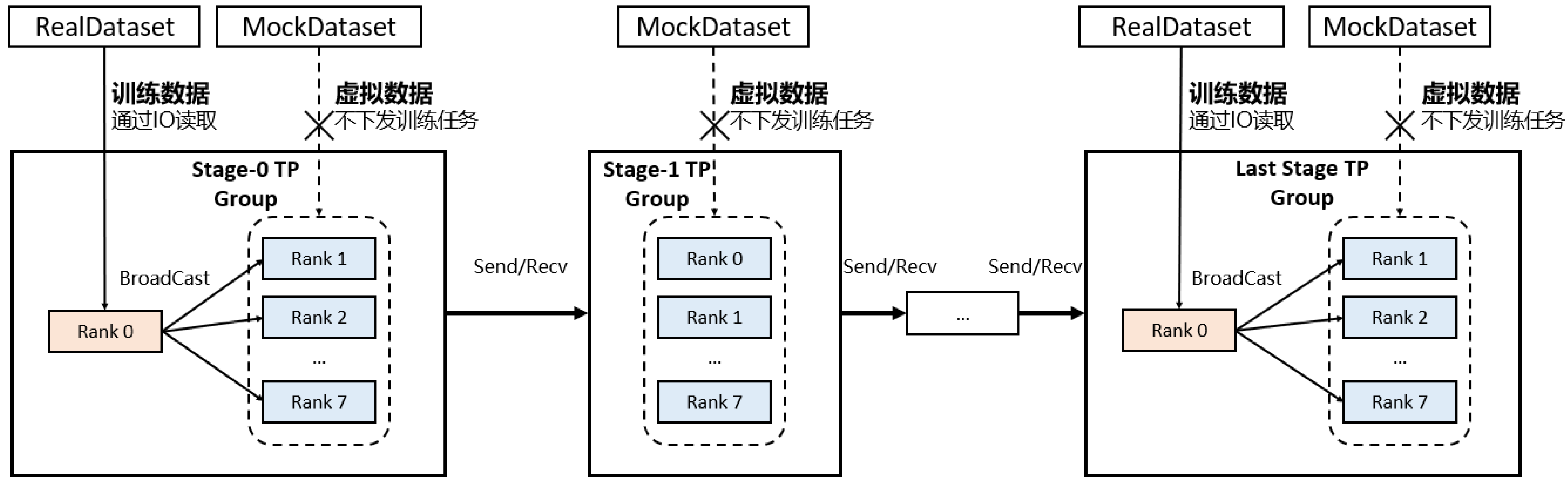
◆HF DataLoader:

支持在线、离线加载Hugging Face社区数据集，主要面向微调场景；

支持用户自定义Handler以及datasets开源接口对数据集进行预处理，内置Alpaca数据格式处理的Handler以及数据集PackingHandler；

MindSpore Transformers数据集功能概述

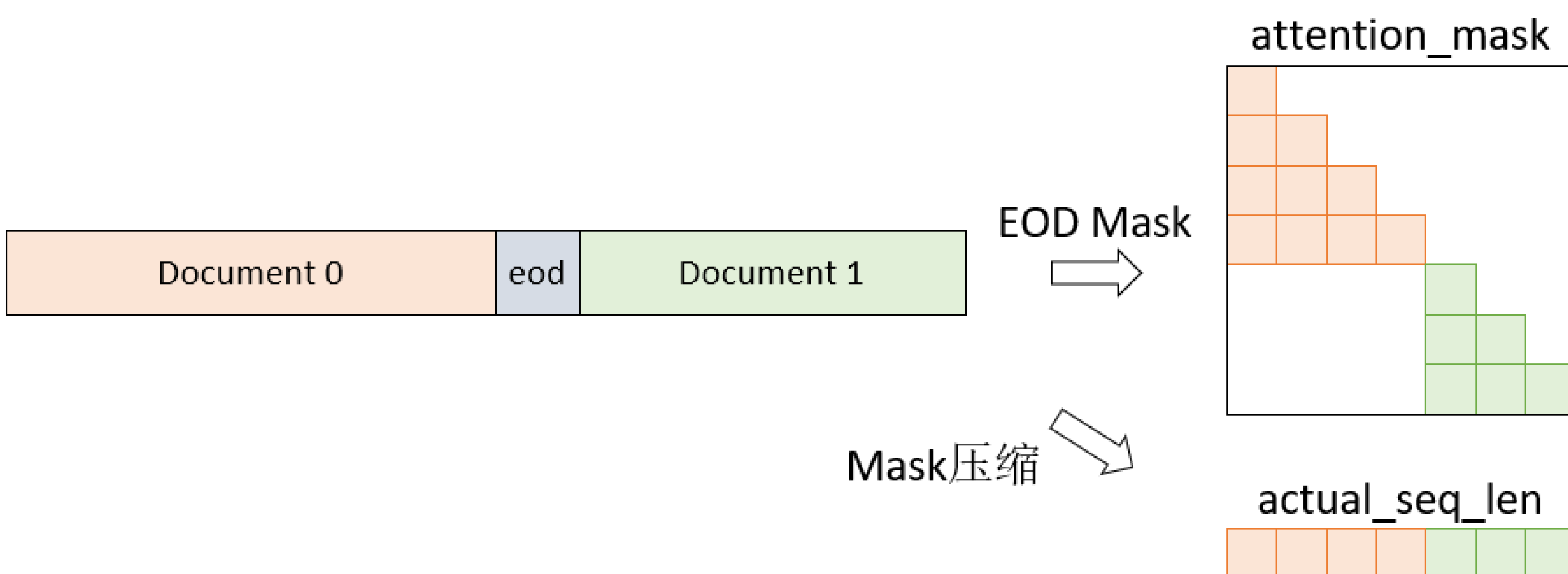
数据通信域去冗余



支持数据通信域去冗余功能：
通过数据广播机制，降低各节点I/O以及Host内存负载；

EOD Mask以及Mask压缩功能：

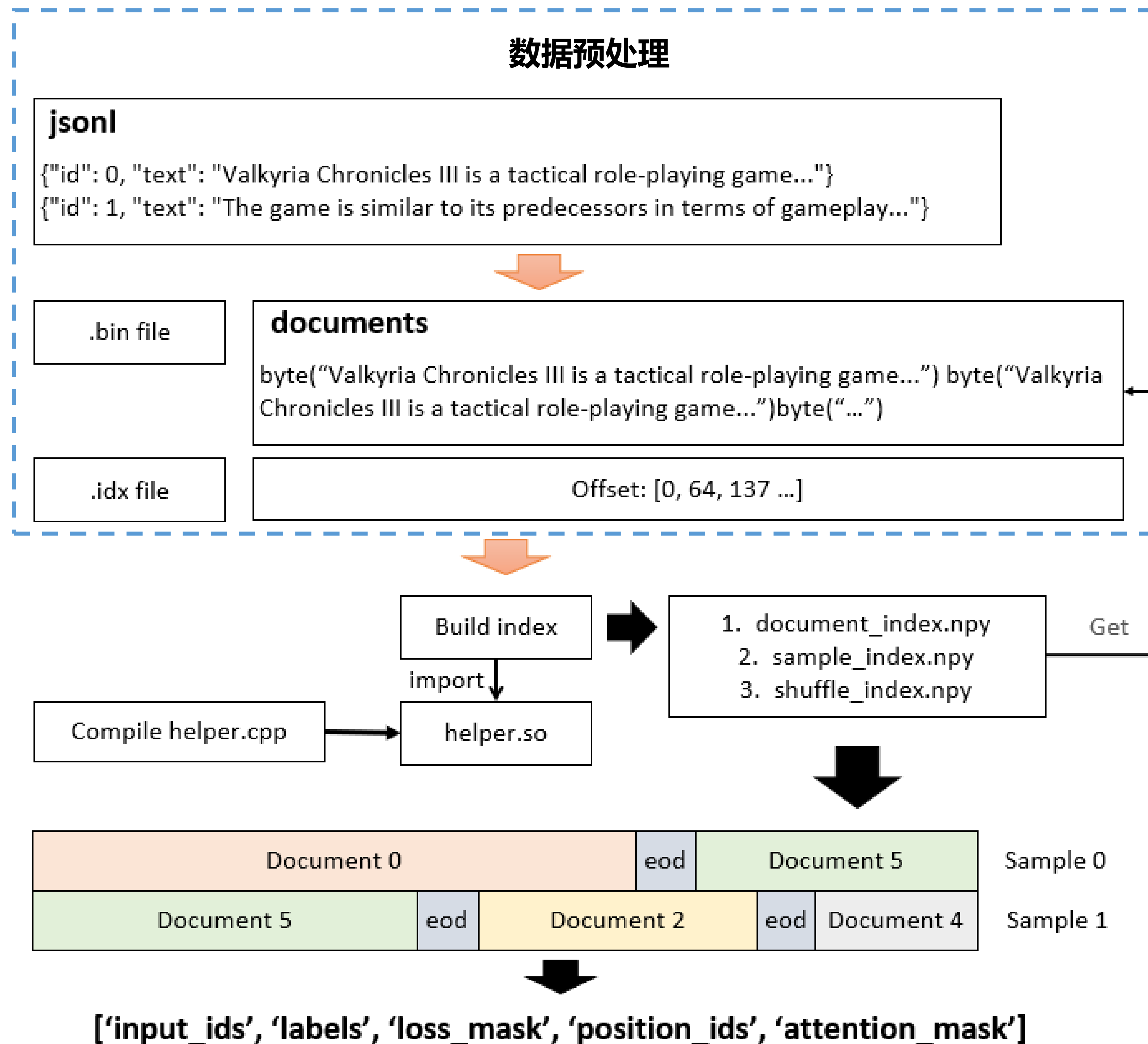
根据eod token的位置返回对应的attention mask或压缩后的mask，支持**32k、128k**长序列训练



数据集	通信域去冗余	EOD Mask	EOD Mask压缩
Blended Megatron DataLoader	√	√	√
HF DataLoader	√	√	√
MindDataset	√	√	-

2. 使用Megatron-LM数据集进行预训练的介绍与实践

使用Megatron-LM数据集进行预训练的介绍与实践



➤ 高效的二进制存取方案，支持大规模数据存储

Megatron采用的bin/idx数据集格式实现了数据的随机高效访问，且二进制存储紧凑，占用空间更小，能够轻松处理TB级数据集，目前大规模应用于大语言模型预训练任务中；

➤ 支持多源数据集混合，低I/O和CPU负载

根据多源数据集预先构建的索引避免了训练时的文本解析与分词，可以大幅降低I/O和CPU开销，**此外还新增了数据通信域去冗余特性，进一步优化性能，降低Host内存负载；**

➤ 支持返回EOD Mask以及Mask压缩

在原生Megatron数据集生成EOD Mask的基础上，**新增返回压缩后的attention mask，支持32k、128k长序列训练；**

➤ 优化的随机采样算法

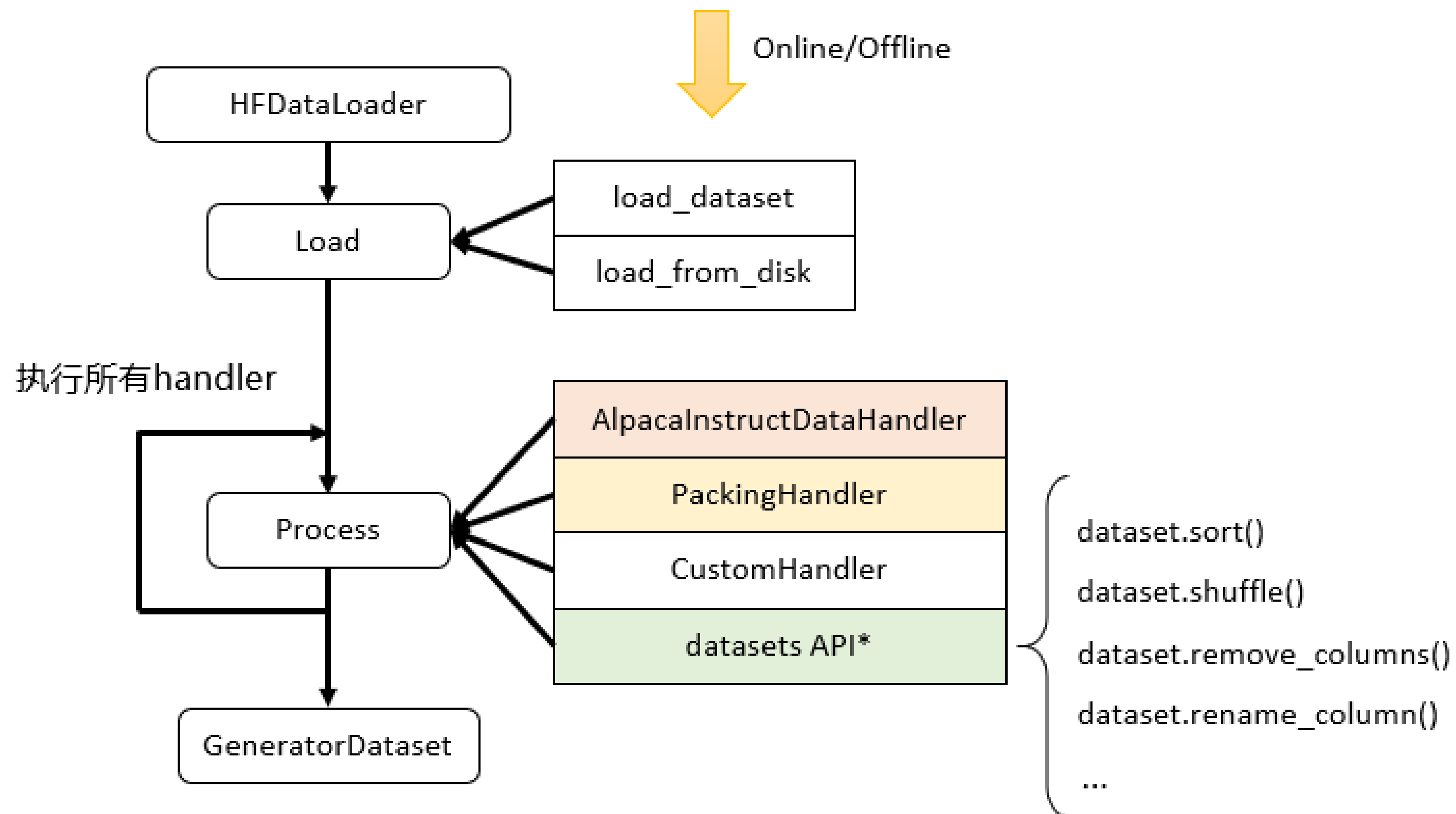
通过任意样本以及序列长度采样和拼接，避免了样本序列的重复，提升模型的训练效果。

[Megatron数据集使用说明文档](#)

3. 使用Hugging Face数据集进行微调的介绍与实践

使用Hugging Face数据集进行微调的介绍与实践

🤗 Hugging Face Datasets 507,186



➤ 支持数据通信域去冗余以及EOD Mask等通用功能

HF DataLoader适配数据集去冗余功能，同时可以在Packing后的数据集上开启EOD Mask以及Mask压缩功能；

[Hugging Face数据集使用说明文档](#)

➤ 兼容Hugging Face社区开源数据集，开箱即用

Hugging Face社区包含数十万开源数据集，涵盖NLP、CV、语音等领域，用户可以方便地分享和复用数据集；

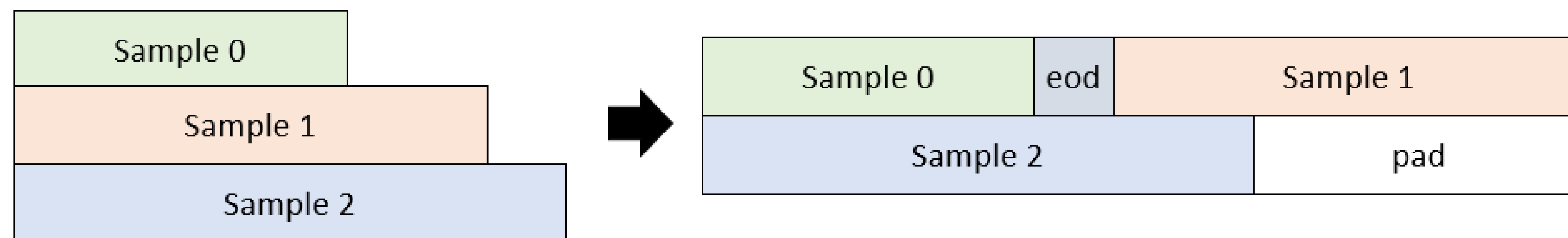
➤ 良好的兼容性，支持多种数据集加载及处理方式

数据集加载方面通过多个接口支持HF数据集在线、离线加载；

数据集处理方面，支持多个原生数据集预处理接口，同时支持用户自定义数据预处理操作，对于多个数据预处理流程可以串行完成，无需多次处理；

➤ 支持数据Packing算法

在使用长序列进行模型微调时，可以通过对数据进行packing，将多个未达到最大长度的数据进行拼接，以提升模型训练效率；



MindSpore Transformers构造自定义Handler

代码实现 – 类

mindformers/dataset/handler/custom_handler.py

```
class CustomHandler:

    def __init__(self, **kwargs):
        self.seq_length = kwargs.get('seq_length', 4096)

    def __call__(self, dataset):
        return dataset.map(self.process)

    def process(self, sample):
        input_ids = sample.get('input_ids')
        input_ids = input_ids[:self.seq_length]
        sample['input_ids'] = input_ids
        return sample
```

mindformers/dataset/handler/__init__.py

```
from .custom_handler import CustomHandler
```

```
train_dataset: &train_dataset
data_loader:
    type: HFDataLoader
    # datasets load arguments
    load_func: 'load_dataset'
    path: "/path/dataset_dir"
    split: "train"
    # dataset process arguments
    handler:
        - type: CustomHandler
          seq_length: 2048
```

代码实现 – 方法

mindformers/dataset/handler/custom_handler.py

```
def custom_process(dataset, **kwargs):
    seq_length = kwargs.get('seq_length', 4096)
    dataset = dataset.map(process, fn_kwargs={'seq_length': seq_length})
    return dataset

def process(sample, seq_length):
    input_ids = sample.get('input_ids')
    input_ids = input_ids[:seq_length]
    sample['input_ids'] = input_ids
    return sample
```

mindformers/dataset/handler/__init__.py

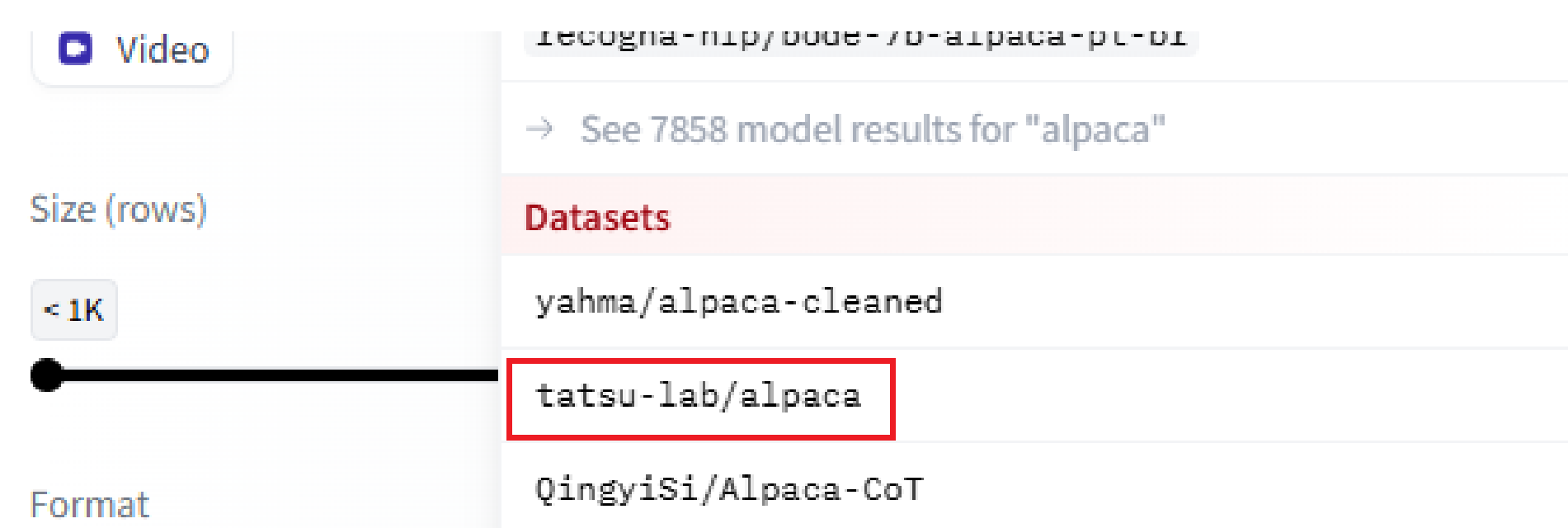
```
from .custom_handler import custom_handler
```

```
train_dataset: &train_dataset
data_loader:
    type: HFDataLoader
    # datasets load arguments
    load_func: 'load_dataset'
    path: "/path/dataset_dir"
    split: "train"
    # dataset process arguments
    handler:
        - type: custom_handler
          seq_length: 2048
```


端到端构造自定义Hugging Face数据集处理流程

Step 1 确定数据集

<https://huggingface.co/datasets>



从Hugging Face社区搜索可用的数据集，框架支持在线和离线加载

Step 3 根据脚本构造配置文件

```
# dataset process arguments
handler:
- type: AlpacaInstructDataHandler
  seq_length: 4096
  padding: False
  tokenizer:
    pretrained_model_dir: "/path/to/Qwen3-32B"
    trust_remote_code: True
    padding_side: 'right'
```

Step 2 构造自定义数据集处理脚本

[mindformers/dataset/handler/alpaca_handler.py](https://github.com/mindformers/mindformers/blob/main/mindformers/dataset/handler/alpaca_handler.py)

注意事项：

1. 如果数据集返回值直接用于模型训练，则返回经过padding到模型seq_length的['input_ids' , 'labels'];
2. 如果数据集后续输入PackingHandler，则返回未经过padding的['input_ids' , 'labels'];
3. 返回数据的 'input_ids' 和 'labels' 需要进行错位处理，即 labels= input_ids[1:] + pad_token，同时设置其中输入的部分为0，表示不计算该token的loss;

4. MindSpore Transformers构造自定义数据集介绍

MindSpore Transformers构造自定义数据集

自定义DataLoader

```
import numpy as np
from mindspore.dataset import GeneratorDataset
from mindformers.tools.register.register import MindFormerRegister, MindFormerModuleType
@MindFormerRegister.register(MindFormerModuleType.DATASET_LOADER)
class CustomDataLoader:
    def __new__(cls,
                seq_length,
                shard_id,
                num_shards,
                column_names,
                size: int = 10240,
                use_actual_seq_len: bool = False,
                **kwargs):
        return GeneratorDataset(
            CustomDataset(seq_length=seq_length, size=size, use_actual_seq_len=use_actual_seq_len),
            column_names=column_names,
            num_parallel_workers=2,
            shuffle=True,
            num_shards=num_shards,
            shard_id=shard_id,
            python_multiprocessing=True,
        )
```

[construct_args_key功能代码示例](#)

自定义Dataset

```
class CustomDataset:
    def __init__(self, seq_length, size: int = 10240, use_actual_seq_len: bool = False):
        self.seq_length = seq_length
        self.size = size
        self.use_actual_seq_len = use_actual_seq_len

    def __getitem__(self, index):
        input_ids = np.ones((self.seq_length,), dtype=np.int32)
        labels = np.ones((self.seq_length,), dtype=np.int32)
        loss_mask = np.ones((self.seq_length,), dtype=np.int32)
        position_ids = np.arange(self.seq_length, dtype=np.int32)
        attention_mask = generate_mask(self.seq_length, self.use_actual_seq_len)

        return (
            input_ids.astype(np.int32),
            labels.astype(np.int32),
            loss_mask.astype(np.int32),
            position_ids.astype(np.int32),
            attention_mask.astype(np.int32)
        )

    def __len__(self):
        return self.size
```

配置文件

```
data_loader:
  type: CustomDataLoader
  seq_length: 4096
  size: 51200
  use_actual_seq_len: False
  input_columns: ["input_ids", "labels", "loss_mask", "position_ids", 'attention_mask']
  construct_args_key: ["input_ids", "labels", "loss_mask", "position_ids", 'attention_mask']
```

与数据集__getitem__接口返回列一致

num_shards和shard_id为框架计算得出，无需配置

自定义接口入参都可以从**kwargs中获取

谢谢观看！

MindSpore Transformers训推Mcore架构介绍

MindSpore Transformers 大模型系列课程